



Accessly

Kraków without barriers

The city. Accessible for you.

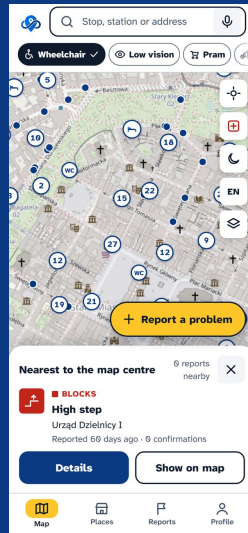
Team Accessly · Team ID: ...

HackYeah 2026 · “Cracow without barriers” challenge

Demo: <https://accessly.example.pl>

Code: <https://github.com/HakerSII/accessly>

Video (3 min): <https://youtu.be/...>



Reliable information instead of guesswork

Problem

- Accessibility data is scattered, incomplete and quickly goes out of date.
- Without a source and a date, users cannot tell whether to trust it.
- Keeping a complete database for a whole city up to date by hand does not scale.

Who it is for

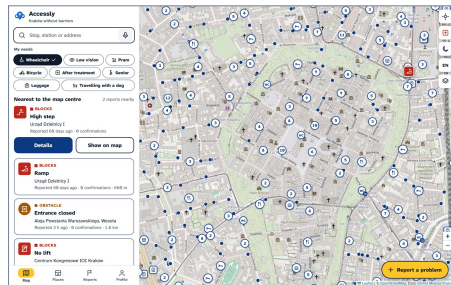
People with access needs and their carers, parents with prams, older people, people recovering from medical procedures, tourists with luggage or pets, and the medical tourism industry.

Accessly

Combines public data, information from venue owners and confirmations from the community. Every piece of information has a source, a date and a verification status.

A working prototype: a web app for phone and desktop

- **6,545 places, 27 accessibility features**
Concrete information about the entrance, lift, toilet, parking and other needs.
- **Matched to the user**
The needs profile filters and ranks places by what really matters.
- **Live, trustworthy data**
Source, date and verification status for every piece of information, plus community confirmations.
- **AI + barrier map**
The assistant recommends places, and users report and update real obstacles.



“Wheelchair” profile: places, layers, reported barriers with severity and confirmations. The list next to the map is its text alternative.

6,545

places from OSM

11

data layers

503

tests, offline

Demo: Anna, a wheelchair user, looks for a café in the centre

	Step	What to look at
1	Needs profile “Wheelchair”	the map and lists show only barriers that matter for a wheelchair; ranking by match
2	Question: “a café in the centre, wheelchair, with a toilet”	a ranking with reasons: what the place meets and what is <i>unknown</i>
3	Place card	every fact has a source, a date and a status; missing data is named as such, with an “Add” button
4	Report “No lift” at a stop, severity “Blocks”	square marker = blocks; “Still there” / “Fixed” votes; the outage expires after 14 days
5	Owner panel: confirm the information	status “Confirmed by the owner”; Anna gets a notification

Edge cases shown in the demo: conflicting data → “Needs re-checking”; city data unavailable → snapshot with an “as of” date; missing information never counts as accessible.

Sample data is marked: demo accounts (anna@, kasia@, admin@accessly.test), test reports, a “Demo mode” label.

Meets 0 of your 3 needs

Does not meet: Step-free entrance, Accessible toilet
No data: Wide doors (at least 90 cm). If you know, add it below or ask the owner.

Step 3: the match summary – what the place meets, what it does not, what is unknown.

 **Step-free entrance** · important to you No

From an official source

Source: OpenStreetMap

02/10/2026

częściowo dostępne (wheelchair=limited)

Up to date

Outdated

Correct

One fact: value, source (OpenStreetMap), date (2 Oct 2026), status and the Up to date / Outdated / Correct buttons.

Data: sources, updates, trustworthiness

Source	What it provides	Updates and fallback when the source is down
OpenStreetMap (ODbL)	6,545 places with attributes from tags (wheelchair, toilets:wheelchair, changing_table...); lifts, kerbs, crossings with tactile paving, luggage storage, A&E and 24-hour pharmacies	Geofabrik extract → build_osm_*.py scripts → snapshot in the repository (as of 2 Oct 2026, date shown in the app); the import never overwrites values from owners and users
Kraków open data (ArcGIS, otwartedane.um.krakow.pl)	stops with platform kerb type (3,756), public toilets (50), disabled parking spaces (2,037), bike routes (2,633), bike stands (3,157), bike infrastructure (199)	fetches in pages of 2,000, 24-hour cache, refreshed in the background; when the service is silent: cache → repository snapshot with an “as of” date → fetch on demand; city attribution on the map
Venue owner	attribute changes, confirmations, photos, points inside the venue (lifts, toilets, entrances)	visible at once as “Confirmed by the owner”; claiming a profile is approved by a moderator
Community	Up to date / Outdated / Correct / Add, reviews, questions, photos, barrier reports and votes	corrections accepted by the owner or a moderator; 2 confirmations = “Confirmed by the community”; every change stays in the history

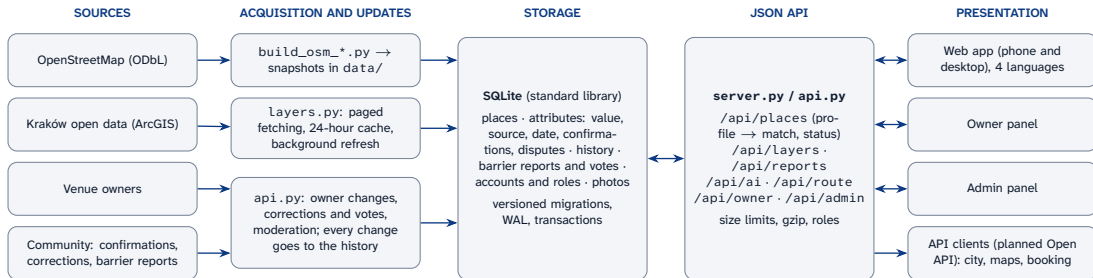
Status of every fact (computed on every read, never stored once):

Confirmed by the owner > Confirmed by the community (≥ 2) > From an official source > Not verified

More “outdated” votes than confirmations, or 365 days without confirmation → Needs re-checking

Rules. Missing information never means “accessible”; conflicting data is flagged, not averaged; the AI assistant never adds facts. Barrier reports expire (permanent 365 days, lift outages 14, closed entrance 7); “still there” extends, 3 “fixed” votes close.

Architecture: data acquisition separated from presentation



Optional services, each with a fallback: Claude API (recommendations; keyword rules without a key) · OpenRouteService (wheelchair routes) · Nominatim (search from the browser, only on submit) · Rampa backend (barrier reports through the `api.js` adapter)

Zero dependencies. Python 3.7+ (standard library only), SQLite, Leaflet; 47 MB Docker image; Ansible + Caddy (HTTPS) on any VPS; 503 tests, no network.

New city. `cities/<city>.json` (name, bounds) + OSM extract + city layers (ArcGIS / WFS) in `Layers.py`; `ACCESSLY_CITY`.

New source, category, attribute. An entry in the catalogue (LAYERS, CATEGORIES, ATTRIBUTES) + labels in 4 languages; consistency tests.

Digital accessibility: WCAG 2.2 AA as the goal

In the prototype (checked on the main scenario)

- **Keyboard:** every action is a button or a link; a “Skip to content” link; visible focus that survives the list refresh every 15 s; Esc closes panels.
- **Screen reader:** landmarks, headings, ARIA labels, short live announcements; the map’s description points to the list; the window title follows the view.
- **Map as text:** the Reports, Places and nearest-layer-items lists; severity by shape and label, not by colour alone.
- **Contrast and legibility:** colour tokens 4.5:1 (text) and 3:1 (controls) checked by a test; night mode; the Atkinson Hyperlegible typeface; 4 languages.
- **Forms and touch:** errors in `role="alert"` naming the field, “Next” is never disabled; targets ≥ 44 px; the layout works at 400 % zoom.

Still to do

- An audit with NVDA / VoiceOver and tests with wheelchair users and people with low vision.
- Keyboard access to map markers (Leaflet); an alternative to placing the pin by panning the map (2.5.7).
- Owner and admin panels to the same standard, and in English.
- An accessibility statement and a conformance report.

The “WCAG 2.2 A” and “WCAG 2.2 AA” iterations are covered by the AccessibilityTests suite.

Privacy, security and operations outside the City

User data and security

- **Without an account:** map, catalogue and barrier reports, anonymously; no IP addresses or device identifiers are stored.
- **The needs profile is a set of preferences** (steps, lift, quiet), never a diagnosis or disability information, exactly as the brief asks.
- **Passwordless account:** e-mail + one-time code (15 min, 5 attempts); codes and tokens stored as SHA-256 hashes; HttpOnly, SameSite, Secure cookie; one-click account deletion.
- **Server:** HTTPS (Caddy, Let's Encrypt), request limits (10 kB, photos 4 MB), user content always escaped (tested), roles checked on every request; privacy policy inside the app.

Launch and operations – proposal

- **Operator:** the Accessly team (company or foundation) or a community partner; the City maintains no database and grants no access to internal systems.
- **Cost (estimate):** one VPS in the EU at about 60–120 PLN (15–30 EUR) a month, a domain, transactional e-mail; the AI assistant is billed per request (fractions of a cent).
- **Updates** with one command (Ansible); city data refreshes itself every 24 h, OSM by script once a month; SQLite backups.
- **Reports and moderation** in the admin panel (queue, claims, corrections); portability: Docker on any infrastructure, open licences listed in the app.

Business model and commercialisation

Segment	Offer	Revenue
Residents and tourists	the app, needs profile, assistant, barrier map	free, always – they create the data
Venue owners (hotels, food & drink, culture, chains)	basic panel free; Pro plan : owner badge, statistics on “what guests look for and what is missing”, bulk facilities, an accessibility widget for the website and the booking system	subscription per venue or a package for chains
Event organisers, property managers	time-limited package: temporary entrances and routes, a barrier map for the event	fee per event / venue
Cities and regions	white-label deployment (configuration, OSM import, city layers); reports on gaps in data and facilities	deployment + annual maintenance
Maps, travel apps, booking systems	Open API with attribution: accessibility facts with source, date and status	API licence / request limits

Principles that protect the value of the data. We do not sell personal data; community content and OSM data stay open (ODbL). Owners pay for tools and visibility, not for a “better” status – the status always follows verification.

From prototype to service

What we have today

- A prototype for phone and desktop: 6,545 places, 27 attributes, 11 layers, barrier map, assistant, 4 languages.
- An owner panel and an admin panel (queue, statistics, data gaps).
- Source, date and status on every fact; missing data shown as missing.
- Checked WCAG 2.2 A/AA criteria on the main scenario; 503 tests, Docker, Ansible, HTTPS.

Plan for the next 3–6 months

- 1 **Pilot** with a disability organisation, 2–3 hotels and cultural institutions; an accessibility audit with users.
- 2 **Production sign-in** (transactional e-mail, optional OAuth), notifications, offline mode.
- 3 **AI for checking photos and text:** suggestions to confirm, never facts; the trust engine from Rampa.
- 4 **Open API and a widget** for owners; integration with booking systems.
- 5 **Second city:** city file + OSM extract + city layers; prerequisite: open city data (or OSM alone) and a local partner for moderation.

Not another static map – a living, verifiable picture of a city's accessibility.

Demo: <https://accessly.example.pl> · Code: <https://github.com/HakerSII/accessly> · Video: <https://youtu.be/...>